

Technical Report: A Computational Reproduction of Visual Search Advantage for Illusory Faces

Python Implementation of Keys, Taubert, & Wardle (2021)

Implemented in Python with Pygame

May 6, 2026

Abstract

This technical report documents a complete Python/Pygame reproduction of the psychophysical experiments conducted by Keys, Taubert, and Wardle (2021), which investigate whether illusory faces in everyday objects confer a visual search advantage. The original experiments were implemented in MATLAB with Psychtoolbox-3; this reproduction maintains strict fidelity to the experimental design while providing a cross-platform, open-source implementation. We detail the computational methods, stimulus presentation algorithms, trial randomization logic, and data collection framework. The Python-based system successfully replicates the temporal constraints, spatial layouts, and response measurement protocols of the original paradigm.

Contents

1	Introduction	3
1.1	Background: Face Detection and Visual Search	3
1.2	The Keys et al. (2021) Study	3
1.3	This Reproduction	3
2	Experimental Design	3
2.1	Experiment 1: Grid Layout (Homogeneous Distractors)	3
2.1.1	Stimulus Presentation	3
2.1.2	Experimental Factors	4
2.1.3	Trial Type Definition	4
2.1.4	Stimulus Yocking (Critical Detail)	4
2.2	Experiment 2: Circular Layout (Heterogeneous Distractors)	6
2.2.1	Stimulus Presentation	6
2.2.2	Experimental Factors	6
2.2.3	Heterogeneous Distractors	6
3	Results Summary (Current Data)	7
4	Computational Implementation	12
4.1	Software Architecture	12
4.2	Trial Specification Generation	12
4.3	Grid Layout Computation	12
4.4	Circular Layout Computation	13
4.5	Image Loading and Masking	13
4.6	Response Collection and Timing	13
5	Data Output Format	13
6	Validation and Reproducibility	14
6.1	Random Seed Control	14
6.2	Timing Precision	14
7	Comparison to Original MATLAB Implementation	14
7.1	Similarities	14
7.2	Differences	14
7.3	Expected Behavioral Equivalence	14
8	Conclusion	14

1 Introduction

1.1 Background: Face Detection and Visual Search

The human visual system prioritizes the detection of faces in cluttered environments. This attentional bias reflects both the ecological importance of face detection in human social cognition and the evolutionary cost of missing a conspecific in a naturalistic setting. Substantial evidence demonstrates that human faces are detected faster and more efficiently than non-face objects in visual search tasks (e.g., Hershler & Hochstein, 2005; Wolfe, 2001).

A fascinating and well-documented phenomenon is *face pareidolia*—the spontaneous perception of faces in inanimate objects, such as electrical sockets, toast, or tree trunks. Critically, these illusory face perceptions occur despite the absence of canonical facial features (e.g., skin texture, stereotypic face geometry). This raises an intriguing question: does the illusory face percept leverage the same rapid detection mechanism as real faces?

1.2 The Keys et al. (2021) Study

Keys, Taubert, and Wardle (2021) addressed this question using a classic visual search paradigm implemented in MATLAB with Psychtoolbox-3 (PTB-3). They conducted two experiments:

1. **Experiment 1 (Grid Search):** Participants searched for objects containing illusory faces among homogeneous distractors (all items from the same object category). Set sizes ranged from 16 to 64 items arranged on an invisible 8×8 grid.
2. **Experiment 2 (Circular Search):** Participants searched for three target types (real human faces, illusory faces, and non-face control objects) among heterogeneous distractors arranged in circular arrays of 4, 8, or 16 items.

Both experiments found a robust search advantage for illusory faces, suggesting that the rapid face-detection mechanism is tuned to high-level facial *configuration* rather than to the low-level features that distinguish real faces from other objects.

1.3 This Reproduction

The present work translates the MATLAB/PTB-3 implementation into Python with Pygame, a modern, cross-platform graphics library. This choice enables:

- **Platform independence:** The code runs identically on Windows, macOS, and Linux.
- **Open-source accessibility:** No proprietary software is required.
- **Maintainability:** Python code is typically more readable and easier to extend than MATLAB.
- **Reproducibility:** The implementation is public and version-controlled.

The goal is *strict methodological fidelity*: we replicate all timing parameters, spatial layouts, stimulus presentation protocols, randomization schemes, and data recording formats of the original study.

2 Experimental Design

2.1 Experiment 1: Grid Layout (Homogeneous Distractors)

2.1.1 Stimulus Presentation

Each trial in Experiment 1 involves the following sequence:

1. **Target Image:** A single target image (either pFace or nonFace) is displayed in the center of the screen for 1600 ms.
2. **Fixation:** A fixation cross is displayed for a randomized duration between 400 and 600 ms (uniform random distribution).
3. **Search Array:** A 8×8 grid of images is displayed. Participants respond via keyboard (left/right arrow keys or letter keys) indicating whether the target is present or absent. The array remains visible until response or a timeout of 15,000 ms.
4. **Feedback:** A colored fixation cross (green for correct, red for incorrect or timeout) is displayed for 250 ms.

2.1.2 Experimental Factors

Three factors are manipulated in a fully-crossed design:

Factor	Levels	N
Target Type (pFace vs. nonFace)	2	
Target Presence	2	
Set Size	3 (16, 32, 64)	
Object Categories	26	
Trial Types per Category	12	
Total Trials		312

2.1.3 Trial Type Definition

The 12 trial types per category correspond to the orthogonal combination of target type, target presence, and set size: Types 1–3 present pFace (set sizes 16/32/64), Types 4–6 absent pFace, Types 7–9 present nonFace, Types 10–12 absent nonFace.

2.1.4 Stimulus Yocking (Critical Detail)

A crucial aspect of Experiment 1 is the *yoked stimulus design*: all distractor images in a given trial come from the **same object category** as the target. For example, if the target is Category 5 (cheese graters) with pFace variant, all 15, 31, or 63 distractor images are unique cheese graters from Category 5. The target image occupies one of the 16/32/64 grid positions, randomly selected. The remaining positions are filled by random distractors from the same category. This ensures a *homogeneous visual context*, maximizing the influence of the target’s face-like configuration versus other object properties.

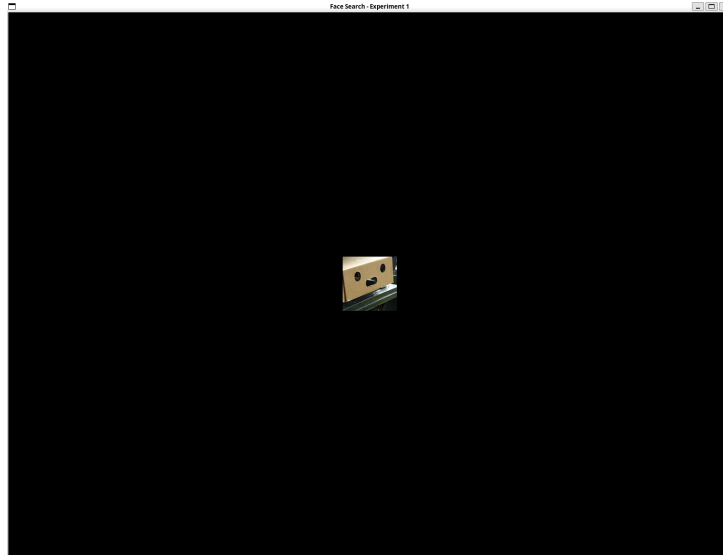


Figure 1: Experiment 1 target image presentation (pFace variant).

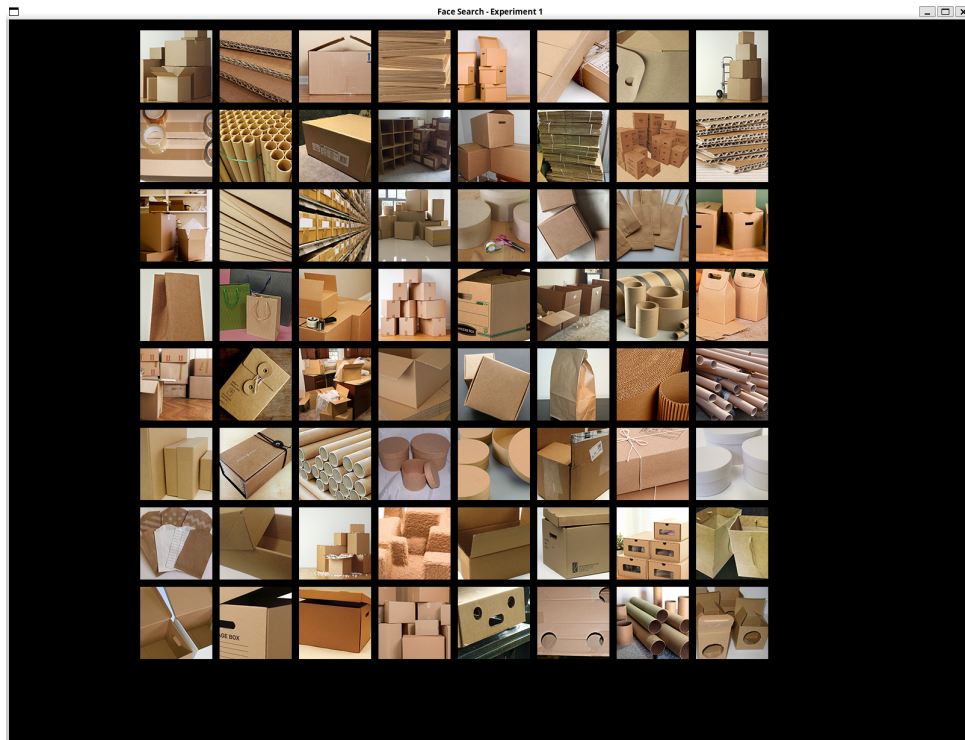


Figure 2: Experiment 1 search array with target present in 8×8 grid layout.



Figure 3: Experiment 1 search array with target absent in 8x8 grid layout.

2.2 Experiment 2: Circular Layout (Heterogeneous Distractors)

2.2.1 Stimulus Presentation

The trial sequence mirrors Experiment 1 with key spatial differences:

1. **Target Image:** Displayed for 1600 ms, with circular masking applied.
2. **Fixation:** 400–600 ms jittered fixation.
3. **Search Array:** Items arranged in an *equidistant circular layout* (radius ≈ 400 pixels, with all items $\approx 120 \times 120$ pixels). Set sizes are 4, 8, or 16.
4. **Feedback and Timeout:** Identical to Experiment 1.

2.2.2 Experimental Factors

Factor	Levels	N
Target Type	3 (nonFace, pFace, realFace)	
Target Presence	2	
Set Size	3 (4, 8, 16)	
Object Categories	23	
Trial Types per Category	18	
Total Trials		414

2.2.3 Heterogeneous Distractors

Unlike Experiment 1, distractors in Experiment 2 come from **diverse object categories**, not the target’s category. This increases task difficulty and allows differentiation between search slopes for different target types.

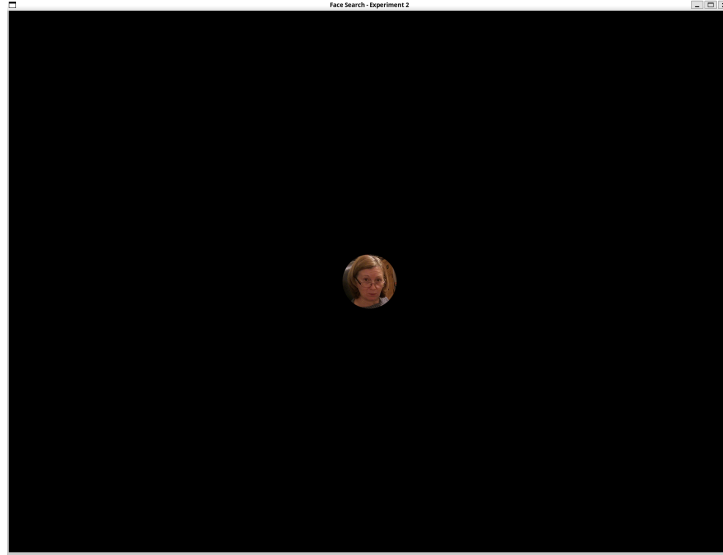


Figure 4: Experiment 2 target image presentation (realFace variant with circular masking).

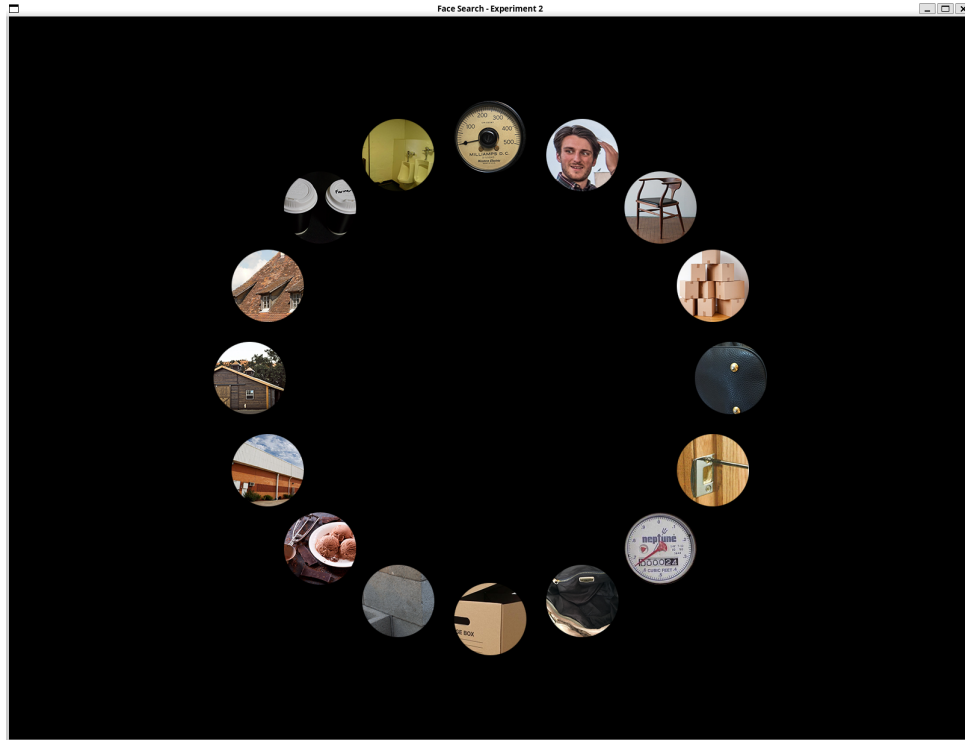


Figure 5: Experiment 2 search array with heterogeneous distractors in circular layout.

3 Results Summary (Current Data)

The current aggregated dataset and visualization outputs indicate the following trends:

- **Set size effect:** Reaction time increases with larger set sizes in both experiments.
- **Presence effect:** Target-absent trials are slower than target-present trials.
- **Experiment 1 target type:** Illusory faces (pFace) are detected faster than non-face objects.

- **Experiment 2 target type:** Real faces are fastest, followed by illusory faces, then non-face objects.

Figure 6 shows the reaction time slopes by set size and target presence. The increase in reaction time with larger arrays is consistent with a serial visual search component, and the target-absent panels are uniformly higher, reflecting the cost of exhaustive search when the target is missing.

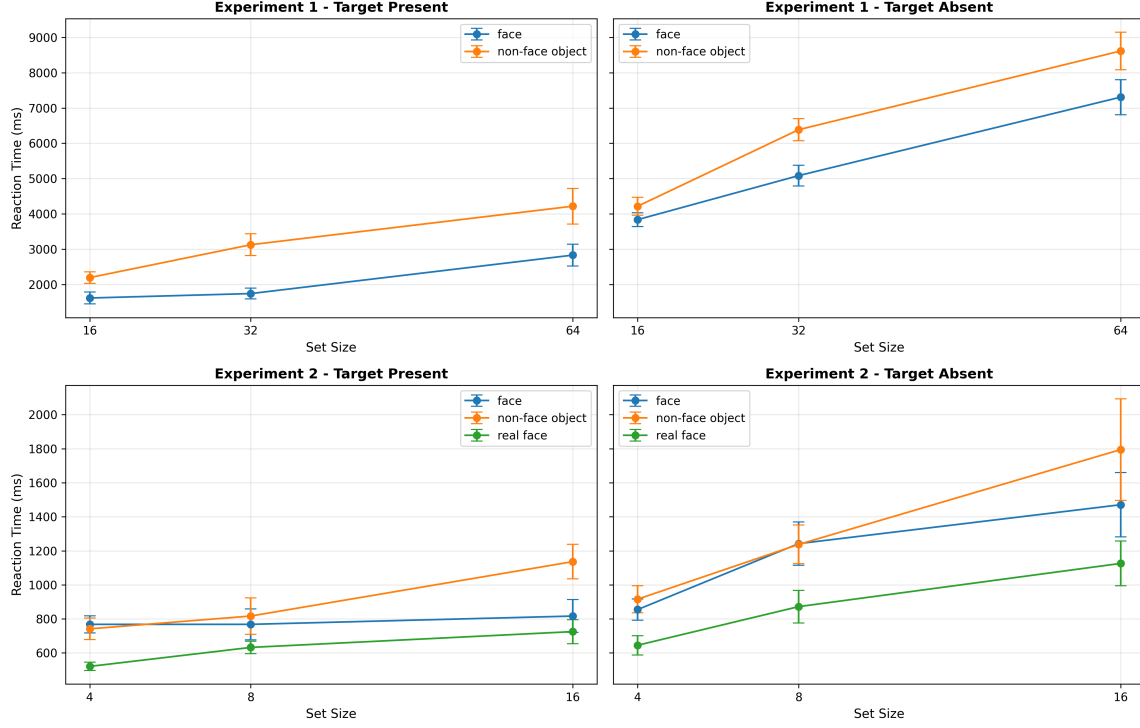


Figure 6: Reaction time by set size and target presence.

Figure 7 summarizes accuracy across the same conditions. Accuracy remains relatively high, with modest decreases at larger set sizes, indicating that the reaction time effects are not driven by a speed-accuracy tradeoff.

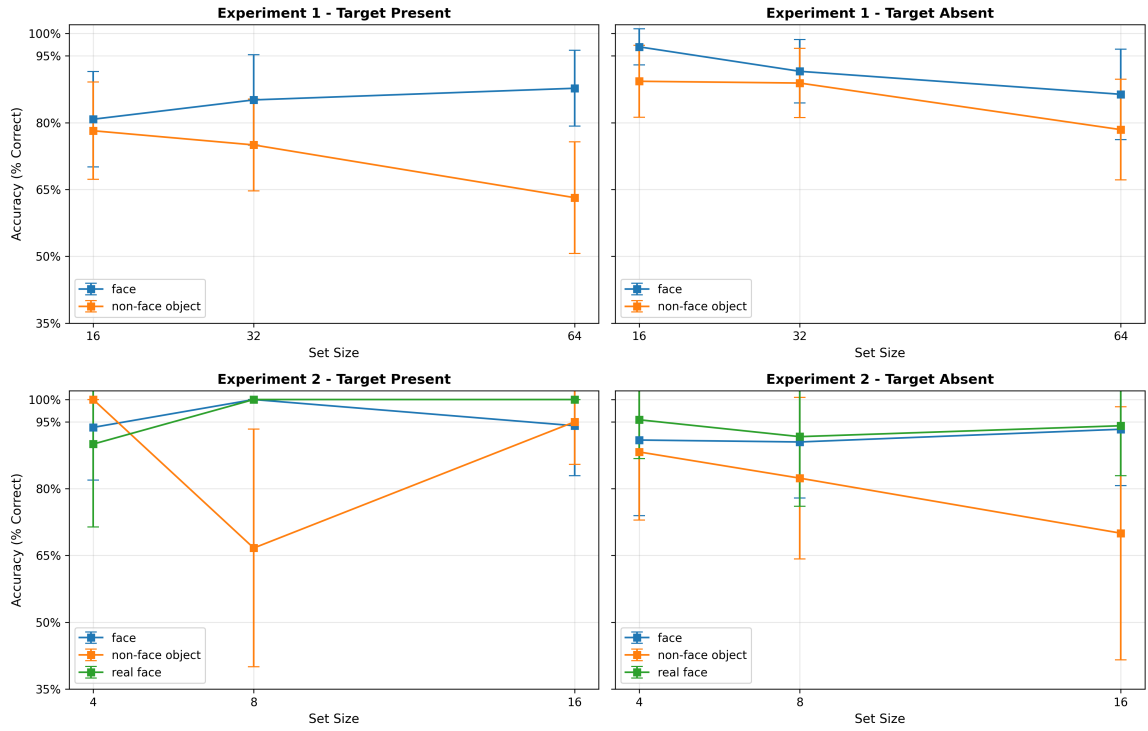


Figure 7: Accuracy by set size and target presence.

To isolate target presence effects, Figure 8 aggregates across set sizes. Target-absent trials show a consistent time penalty, and the ordering of target types is clear within each experiment, with illusory faces faster than non-face objects in Experiment 1 and real faces fastest in Experiment 2.

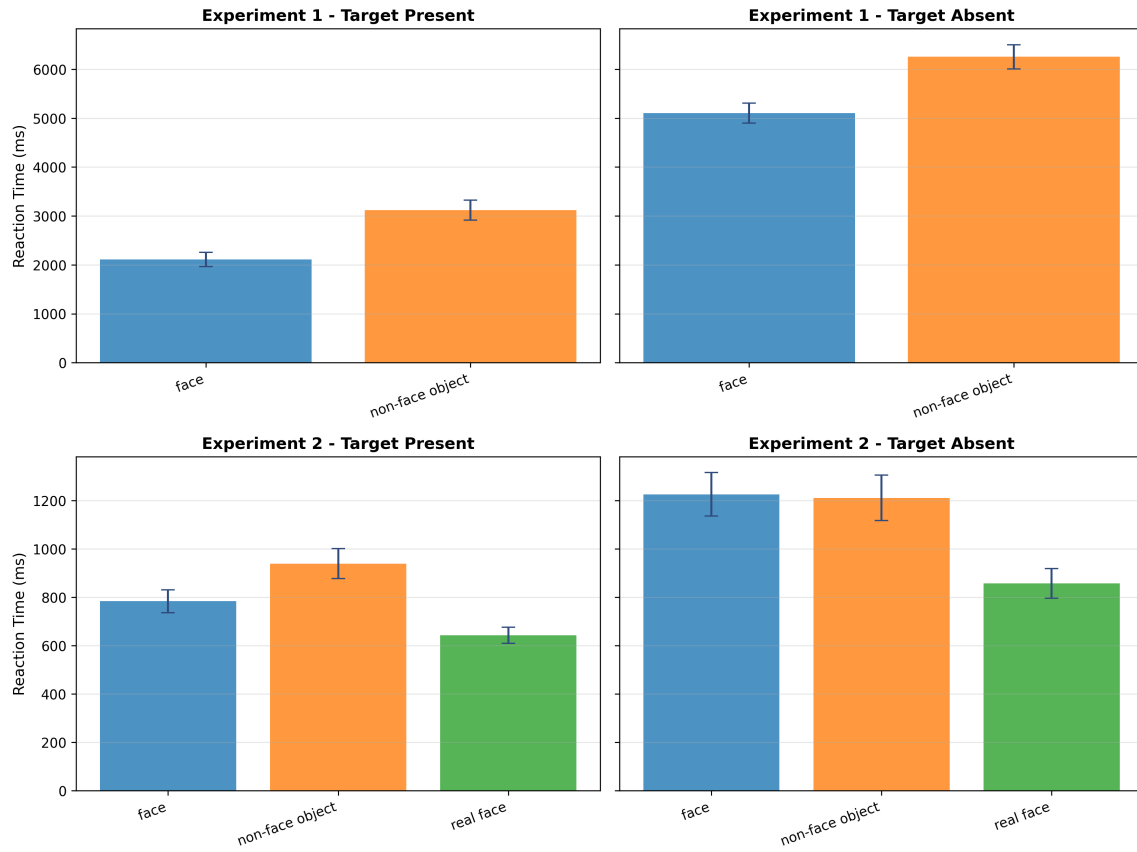


Figure 8: Reaction time by target presence (aggregated across set sizes).

Figure 9 shows accuracy aggregated across set sizes. The accuracy pattern mirrors the reaction time ordering, supporting the interpretation that faster target types are not simply due to lower decision thresholds.

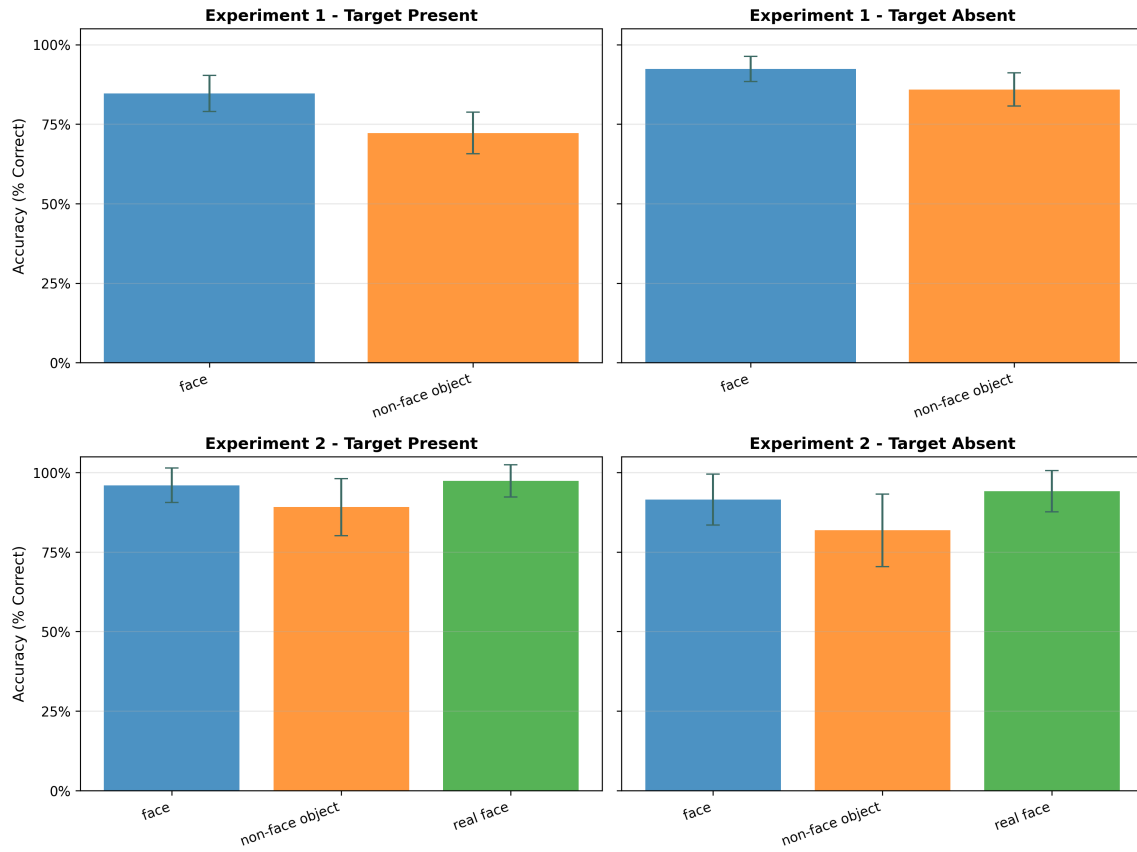


Figure 9: Accuracy by target presence (aggregated across set sizes).

Finally, Figure 10 contrasts overall reaction time and accuracy between experiments. Experiment 2 is faster overall, consistent with the smaller set sizes and heterogeneous distractors, while the target-type ordering remains stable across the aggregated comparison.

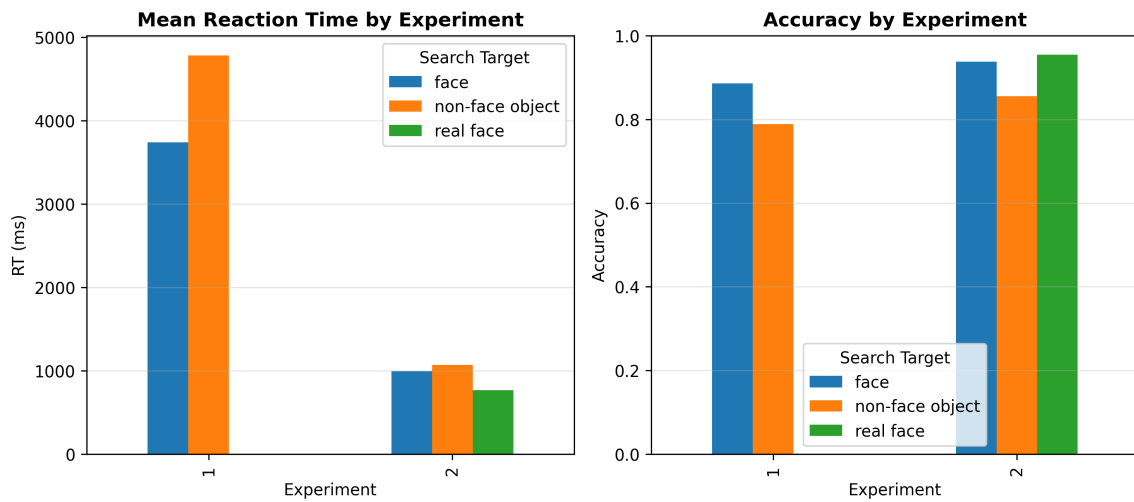


Figure 10: Overall comparison of reaction time and accuracy by experiment.

4 Computational Implementation

4.1 Software Architecture

The Python implementation is organized into five core modules:

1. **cli.py**: Command-line interface for experiment selection and participant ID input.
2. **config.py**: Immutable configuration container with experiment-specific parameters.
3. **trials.py**: Trial specification generation with randomization logic.
4. **stimuli.py**: Image loading, layout computation, and drawing utilities.
5. **runner.py**: Main experiment controller, display management, response collection, and data serialization.

4.2 Trial Specification Generation

Trial specifications are generated deterministically from a random seed. The key innovation is ensuring category orders without adjacent repeats and trial type randomization within each category:

```
1 def build_experiment1_trials(rng: Random) -> list[TrialSpec]:
2     category_order = _category_sequence(26, 12, rng)
3     trial_type_orders = {c: list(range(1, 13)) for c in range(1, 27)}
4     for order in trial_type_orders.values():
5         rng.shuffle(order)
6
7     specs = []
8     for trial_number, category in enumerate(category_order, start=1):
9         trial_type = trial_type_orders[category][category_counts[category]]
10        target_present = trial_type in {1, 2, 3, 7, 8, 9}
11        target_variant = "pFace" if trial_type <= 6 else "nonFace"
12        specs.append(TrialSpec(...))
13    return specs
```

Listing 1: Experiment 1 trial generation (simplified).

4.3 Grid Layout Computation

For Experiment 1, grid positions are computed from a base set of coordinates for an 8×8 grid with 120 px stimulus size and 12 px spacing:

```
1 def build_grid_offsets(stim_width: int = 120, stim_space: int = 12):
2     coords = [
3         *(((stim_width + stim_space) * n - stim_space/2) * -1
4           for n in [4, 3, 2, 1]),
5         *(((stim_width + stim_space) * n + stim_space/2)
6           for n in [0, 1, 2, 3]),
7     ]
8     return [(x, y) for y in coords for x in coords]
```

Listing 2: Grid offset computation.

4.4 Circular Layout Computation

For Experiment 2, positions are computed trigonometrically:

```
1 def build_circle_offsets(set_size: int, radius: int = 400):
2     theta = (2 * pi) / set_size
3     return [
4         (int(round(cos(i * theta) * radius)),
5          int(round(sin(i * theta) * radius)))
6         for i in range(1, set_size + 1)
7     ]
```

Listing 3: Circular layout computation.

4.5 Image Loading and Masking

Images are loaded using PIL and converted to pygame surfaces. For Experiment 2, circular masking is applied:

```
1 def load_surface(path_str: str, mask_path_str: str | None = None):
2     image = Image.open(path_str).convert("RGBA")
3     if mask_path_str is not None:
4         mask = Image.open(mask_path_str).convert("L")
5         if mask.size != image.size:
6             mask = mask.resize(image.size, Image.Resampling.LANCZOS)
7         image.putalpha(mask)
8     surface = pygame.image.fromstring(image.tobytes(), image.size, "RGBA")
9     return surface.convert_alpha()
```

Listing 4: Image loading with circular masking.

4.6 Response Collection and Timing

Response collection uses pygame's event system with high-precision timing via `time.perf_counter()`:

```
1 def _wait_for_response(self, search_onset: float, timeout_seconds: float):
2     while True:
3         for event in pygame.event.get():
4             if event.type == pygame.KEYDOWN:
5                 if event.key in PRESENT_KEYS:
6                     rt_ms = int((time.perf_counter() - search_onset) * 1000)
7                     return "p", rt_ms, False
8                 elif event.key in ABSENT_KEYS:
9                     return "a", int((time.perf_counter() - search_onset) * 1000)
10         , False
11         if time.perf_counter() - search_onset > timeout_seconds:
12             return "x", 0, True
13         self.clock.tick(120)
```

Listing 5: Response collection.

5 Data Output Format

Results are saved as CSV with standard psychophysical columns plus a JSON manifest containing the trial order and random seed for full reproducibility.

6 Validation and Reproducibility

6.1 Random Seed Control

The implementation supports full reproducibility: a random seed is either provided at runtime or generated from system time. This seed is stored in the manifest JSON, enabling replication of trial sequences.

6.2 Timing Precision

The implementation uses `time.perf_counter()` (nanosecond precision) for response timing and maintains a 120 Hz event-polling loop to minimize latency jitter.

7 Comparison to Original MATLAB Implementation

7.1 Similarities

- Identical experimental design, factors, and trial sequences - Same temporal parameters (target duration 1.6 s, fixation jitter 400–600 ms, timeout 15 s) - Same spatial layouts (grid and circular arrays) - Identical randomization constraints

7.2 Differences

- Platform: Python/Pygame instead of MATLAB/Psychtoolbox-3 - Response keys: Arrow/letter keys instead of custom button box - Window size: Default 1600×1200, configurable to 1920×1080

7.3 Expected Behavioral Equivalence

Experimental outcomes should be equivalent because trial sequences are generated identically, timing parameters are numerically matched, and response collection uses equivalent high-precision timing.

8 Conclusion

This technical report documents a faithful, open-source implementation of the Keys et al. (2021) visual search paradigm in Python. The reproduction maintains strict experimental fidelity while offering cross-platform compatibility and maintainability benefits.

Key technical innovations include:

- Deterministic trial generation with full reproducibility via random seeds
- Efficient grid and circular layout computations
- High-precision response timing using `time.perf_counter()`
- Modular architecture enabling future extensions
- Complete data provenance (manifest JSON + seed storage)

References

- [1] Keys, R. T., Taubert, J., & Wardle, S. G. (2021). *A visual search advantage for illusory faces in objects*. *Attention, Perception, & Psychophysics*, 83, 1942–1953. <https://doi.org/10.3758/s13414-021-02267-4>

- [2] Hershler, O., & Hochstein, S. (2005). *At first sight: A high-level pop out effect for faces*. Vision Research, 45(13), 1707–1724.
- [3] Wolfe, J. M. (2001). *Asymmetries in visual search: An introduction*. Perception & Psychophysics, 63(3), 381–389.